

Servlet 乱码问题 & 路径问题 图解

配套《第五章 Servlet》笔记 10.1 / 10.2 节 · 通俗重讲版 · 2026-09

🔒 乱码问题一句话

数据是谁写的（编码），就必须让读的人（解码）用同一套字符集。
乱码 = 写和读用了不同的字符集，没有别的原因。

📁 路径问题一句话

相对路径：从"当前地址栏 URL 的所在目录"出发找目标。

绝对路径：以 / 开头，从 协议+主机+端口 出发找目标。

一、乱码问题

1.1 核心原理：数据在传输中要"编码 / 解码"好几次

整个 Web 请求过程中，数据要经过好几手，每一手都可能"编码"（把文字变成字节）或"解码"（把字节还原成文字）一次：

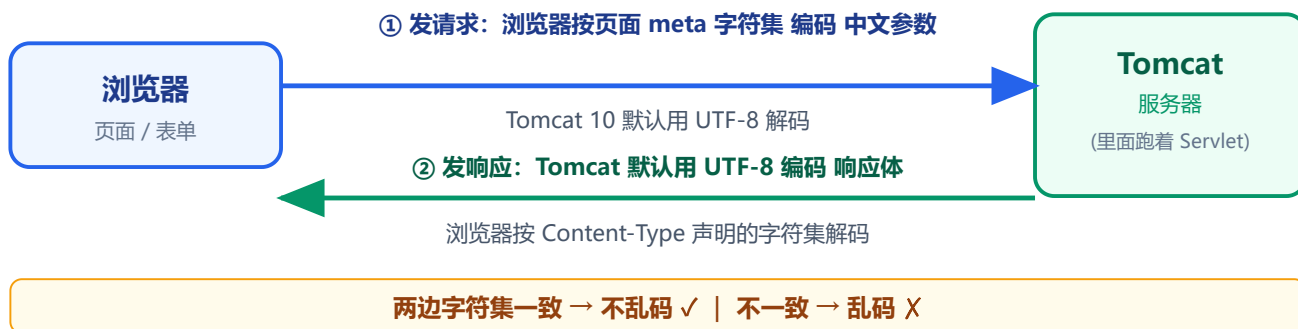
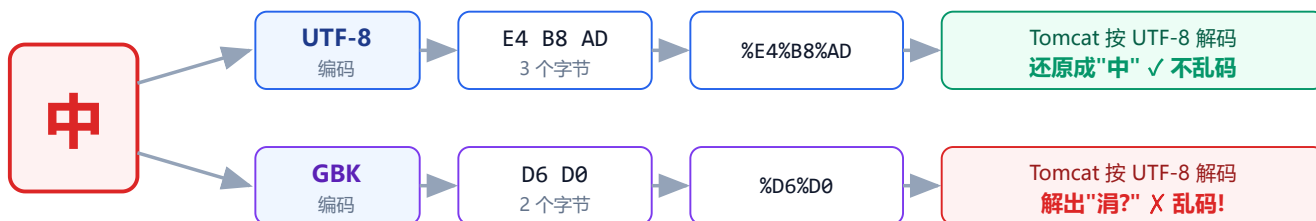


图 1：乱码的唯一原因——相邻两手的字符集不一致

1.2 背景知识：为什么英文从来不乱、中文才乱？

- 所有常见字符集（ASCII、GBK、UTF-8）都兼容 ASCII，所以英文字母、数字、符号永远不乱码。
- 中文不一样：UTF-8 里一个汉字占 3 个字节，GBK 里占 2 个字节。同一个"中"字编出来的字节完全不同。



同一个汉字，不同字符集 → 不同的字节序列 → 拿错的字符集去解，就变成鬼画符

图 2：同一个"中"字的两种编码结果（%XX 形式叫 URL 编码 / 百分号码）

1.3 五大乱码场景总览（每个都问三个问题：谁编码？谁解码？一致吗？）

场景	谁在编码	谁在解码	解决办法（推荐打★）
① HTML 页面乱码	IDEA 保存文件时（项目编码）	浏览器（看 <code><meta charset></code> ）	★ 文件保存为 UTF-8 + meta 写 UTF-8
② 控制台乱码	Tomcat 打日志（默认 UTF-8）	Windows 终端（默认 GBK）	★ <code>logging.properties</code> 里把 UTF-8 改成 GBK，迁就终端
② 附 sout 乱码	javac 编译 .class（UTF-8）	JVM 运行时（默认编码可能不同）	★ 给 Tomcat 的 VM 加参数 <code>-Dfile.encoding=UTF-8</code>
③ GET 参数乱码	浏览器（受页面 meta 影响）	Tomcat 10（默认 UTF-8）	★ 把页面 meta 设为 UTF-8（不推荐改 Tomcat 的 <code>URIEncoding</code> ）
④ POST 参数乱码	浏览器（受页面 meta 影响）	Tomcat 10（默认 UTF-8）	★ 把页面 meta 设为 UTF-8（统一比事后修更省事）
⑤ 响应乱码	你的代码写响应体（默认 UTF-8）	浏览器（没人告诉它，只能猜）	★ <code>resp.setContentType("text/html;charset=UTF-8")</code>

1.4 逐个击破

① **HTML 页面乱码**：文件本身是什么编码保存的，要和 `<meta charset="UTF-8">` 声明的一致。meta 的作用就是“告诉浏览器用什么字符集读这个文件”。IDEA 里把 Global Encoding / Project Encoding 都设为 UTF-8。

② **Tomcat 控制台乱码**：Tomcat 用 UTF-8 打印日志，Windows 黑窗口默认用 GBK 显示 → 乱。把 `tomcat/conf/logging.properties` 里的 UTF-8 全改成 GBK 即可。

② 附：**sout 乱码**：这是另一条链路——javac 编译 .class（UTF-8）→ JVM 加载运行。给运行 Tomcat 的 JVM 加上 `-Dfile.encoding=UTF-8`（IDEA 的 Run Configuration → VM options），让 JVM 解读 .class 里的字符串也用 UTF-8。本质还是那句话：**编译编码和运行解码要一致**。

③ GET 请求乱码（重点）

表单用 GET 提交时，参数拼在 URL 后面，浏览器会按当前页面 meta 声明的字符集把中文做 URL 编码（变成 %XX 百分码）。而 Tomcat 10 默认统一用 UTF-8 解析 URI。

所以：页面 meta 是 GBK → 浏览器用 GBK 编码 → Tomcat 用 UTF-8 解码 → 乱码（见图 2 下分支）。

页面 meta 是 UTF-8 → 两边一致 → 正常。

为什么不推荐改 Tomcat 的 `URIEncoding`？因为项目部署到别人的服务器上，你改不了人家的配置。让自己的页面统一 UTF-8 才是一劳永逸。

④ **POST 请求乱码**：和 GET 同一个原理，只是参数不在 URL 上，而是放在**请求体**里。请求体的字符集同样受页面 meta 影响，Tomcat 10 默认也用 UTF-8 解析请求体。

解决办法一模一样：**meta 设为 UTF-8，从源头统一**。（课程里的"方式2 在 getParameter 之前 req.setCharacterEncoding()"只对请求体有效，且要写在取参数之前，容易忘，所以不推荐。）

⑤ **响应乱码（重点）**：你写的中文响应体默认按 UTF-8 变成字节，但浏览器不知道该用什么字符集读，猜错了就乱。**解决办法：主动告诉它——**

```
resp.setContentType("text/html;charset=UTF-8"); // 必须写在 getWriter() 之前！  
resp.getWriter().write("你好，世界");
```

这一行同时干了两件事：告诉浏览器"① 这是 html 文档 ② 用 UTF-8 读"。

□ 诀

全链路统一 UTF-8：**文件存 UTF-8、meta 写 UTF-8、响应声明 UTF-8。**

做到这三条，九成乱码根本不会发生。老师课上是在演示"怎么制造乱码"来反推原理，你只要记住正确写法就行。

二、路径问题

2.1 两种路径的定义

- **相对路径**：不以 `/` 开头，从"当前资源所在的路径"出发找目标。可以用 `./`（当前目录，可省略）、`../`（上一层，每个抵消一层目录）。
- **绝对路径**：以 `/` 开头，从固定基准（协议+主机+端口，即 `http://localhost:8080`）出发，和当前资源在哪无关。好处：无论页面放哪，写法都一样。

铁律：相对路径认"地址栏 URL"，不认文件在磁盘/项目里的物理位置！ "所在路径" = 地址栏 URL 去掉最后的文件名。这是所有路径陷阱的根源。

2.2 相对路径怎么算：就一个动作——拼接

当前资源所在路径（URL 去掉文件名）+ 你写的相对路径，拼在一起，`../` 抵消一层目录。

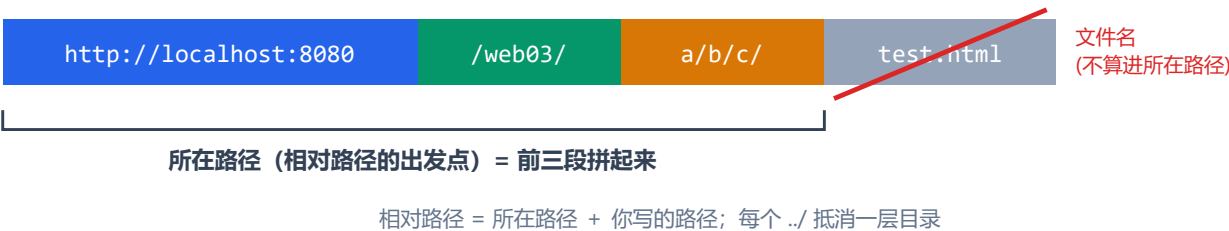


图 3：所在路径 = 地址栏 URL 去掉文件名

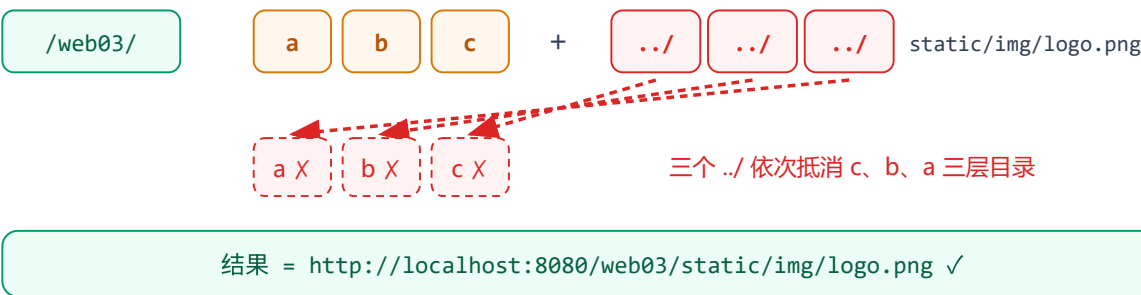


图 4：`../` 抵消法算相对路径（`a/b/c/test.html` 引用 `static/img/logo.png` 的例子）

2.3 前端页面三种情况（同一张图片，三种写法）

情况	地址栏 URL	所在路径	img 的 src 写法
1 页面在 web 根目录 <code>web/index.html</code>	<code>.../web03/index.html</code>	<code>.../web03/</code>	<code>static/img/logo.png</code>
2 页面在三层子目录 <code>web/a/b/c/test.html</code>	<code>.../web03/a/b/c/test.html</code>	<code>.../web03/a/b/c/</code>	<code>../../../../static/img/logo.png</code>
3 页面在 WEB-INF 里 经 <code>view1Servlet</code> 转发	<code>.../web03/view1Servlet</code> 地址栏是Servlet!	<code>.../web03/</code>	<code>static/img/logo.png</code> ← 文件藏得深，但 URL 在根上

情况 3 最反直觉：文件物理上在 WEB-INF/views/ 很深的地方，但 WEB-INF 受保护必须靠 Servlet 转发展示，地址栏停在 Servlet 的 URL 上，所以所在路径算出来是**项目根**，一个 ../ 都不用写。

2.4 绝对路径：要不要写上下文路径，取决于“谁来解析这条路径”

绝对路径以 `/` 开头，但基准是哪，取决于这条路径最终**由谁发请求 / 谁来解析**。这是整个路径问题的核心中的核心：



2.6 转发后目标页面里的相对路径（地址栏陷阱，10.2.3.3）

浏览器地址栏（转发后）

不变，浏览器根本不知道发生了转发）`http://localhost:8080/web03/x/y/servletB` 所以浏览器认为“当前所在路径” = `/web03/x/y/` 页面里所有相对路径都从 `/x/y/` 出发算 实际返回的文件是 `web/a/b/c/test.html` 它在项目里的物理位置（三层深） 相对路径不看物理位置！ `servletB` --转发(服务器内部)--> `test.html` 所以 `test.html` 里引图片要按 `/x/y/` 往回找，需要 3 层 `../` `` ✓

图 6：地址栏陷阱——这就是“转发后页面里图片老加载不出来”的原因

2.7 base 标签：给整页相对路径“改个家”

问题：页面里一堆相对路径，如果页面被放在不同深度的 URL 下，每个都得数着 `../` 改，很累。

base 标签用法

```
<head>
  <base href="/web03/">
</head>
<body>
  <!-- 写的是相对路径，但不再按地址栏算，一律拼到 base 的 href 后面 -->
   ← 实际请求 /web03/static/img/logo.png
</body>
```

- 只对**相对路径**生效，`/` 开头的绝对路径不受影响；
- 相对路径若以 `./` 或 `../` 开头，base 也管不着（所以用 base 的页面里别写 `../`）；
- 弊端：href 里写死了上下文路径 `/web03/`，部署环境一变全得改 → 于是引出 2.8 的办法。

2.8 上下文路径会变：两个终极解法

`/web03_war_exploded` 是部署时生成的名字，换个环境就变。两种解法：

方法1：动态获取上下文路径拼上去（重定向场景）

```
resp.sendRedirect(getServletContext().getContextPath() + "/a/b/c/test.html");
```

方法2（推荐）：部署时把应用上下文路径设为缺省的 `/`（IDEA 部署配置里改）

```
resp.sendRedirect("/a/b/c/test.html"); // 从此所有绝对路径直接 / 开头，一劳永逸
 // 前端同样受益
```

这也解释了后面日程管理案例里为什么敢直接写 `resp.sendRedirect("/registSuccess.html")`——因为项目上下文已经设成 `/` 了。

- 口 诀
- ① 相对路径看地址栏，不看文件位置；`../` 抵消一层。
 - ② 绝对路径 / 开头：浏览器发请求的（前端、重定向）要带上下文，服务器内部转的（转发）不带。
 - ③ 怕上下文变？`getContextPath()` 动态拼，或把上下文设成 `/`，一劳永逸。

三、总结 & 自测

问题	本质	一句话解法
乱码	编码方和解码方字符集不一致	全链路统一 UTF-8；响应主动声明 Content-Type
路径	没想清楚"基准点在哪、谁来解析"	优先绝对路径；先问是浏览器解析还是服务器解析

自测 1： 页面地址栏是 `http://localhost:8080/web03/x/y/z/servletA` 转发来的，页面里要引 `static/img/logo.png`，`src` 怎么写？
答：`../../static/img/logo.png`（所在路径是 `/web03/x/y/z/`，抵消 `x`、`y`、`z` 三层）

自测 2： 在 Servlet 里分别用重定向和转发跳到 `a/b/c/test.html`，两条绝对路径怎么写（上下文为 `/web03`）？
答：重定向 `sendRedirect("/web03/a/b/c/test.html")` 要带上下文；转发 `getRequestDispatcher("/a/b/c/test.html")` 不带。

自测 3： 表单提交中文乱码，后端一切正常，最先检查什么？
答：页面的 `<meta charset>` 是不是 UTF-8（GET 和 POST 的参数编码都受它影响，Tomcat 10 默认按 UTF-8 解）。